

NAME: Atchaya Vharsne S

REG. NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: Operating Systems

CO2 AT3 DEBUGGING OR OPTIMIZATION TASK

1. Deadlock Debugging & Analysis

Problem Analysis

The deadlock occurs because both processes acquire the resources in a different order.

- P1 locks R1 first and then waits for R2.
- P2 locks R2 first and then waits for R1.

Suppose:

- P1 locks R1.
- P2 locks R2.
- P1 waits for R2.
- P2 waits for R1.

Neither process can continue.

This creates a Circular Wait, one of the four Coffman conditions for deadlock.

Optimization

Always acquire resources in the same order.

Both processes should lock R1 first and then R2.

This removes circular wait.

Debugging Program (C)

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t R1, R2;

void *P1(void *arg){

    pthread_mutex_lock(&R1);

    printf("P1 locked R1\n");

    sleep(1);

    pthread_mutex_lock(&R2);

    printf("P1 locked R2\n");

    printf("P1 Critical Section\n");

    pthread_mutex_unlock(&R2);

    pthread_mutex_unlock(&R1);

    return NULL;

}

void *P2(void *arg){

    pthread_mutex_lock(&R1);

    printf("P2 locked R1\n");

    sleep(1);

    pthread_mutex_lock(&R2);

    printf("P2 locked R2\n");

    printf("P2 Critical Section\n");

    pthread_mutex_unlock(&R2);
```

```
    pthread_mutex_unlock(&R1);

    return NULL;
}

int main(){

    pthread_t t1,t2;

    pthread_mutex_init(&R1,NULL);

    pthread_mutex_init(&R2,NULL);

    pthread_create(&t1,NULL,P1,NULL);

    pthread_create(&t2,NULL,P2,NULL);

    pthread_join(t1,NULL);

    pthread_join(t2,NULL);

    return 0;
}
```

Q2. CPU Utilisation Problem

Debugging

Scheduler Log

Ready → Waiting

Waiting → Ready

Ready → Waiting

The processes spend most of their time waiting for **I/O operations**.

Therefore,

- CPU remains idle.
- CPU utilization becomes only 30%.

This is called an I/O-bound workload.

Optimization

1. Increase degree of multiprogramming.
2. Use asynchronous I/O or DMA.
3. Improve scheduling to execute another ready process while one waits.

Program

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main(){
```

```
    int i;
```

```
    for(i=1;i<=5;i++){
```

```
        printf("CPU Executing Process %d\n",i);
```

```
        printf("Process waiting for I/O...\n");
```

```
        sleep(1);
```

```
    printf("Process Ready Again\n\n");  
}  
  
return 0;  
}
```

Q3. Memory Bottleneck Debugging

Memory Analysis

Code = 5 MB

Heap = 150 MB

Stack =

25×1 MB

= 25 MB

Total memory used

$5 + 150 + 25 = 180$ MB

Although memory usage is below 2 GB, many threads access memory simultaneously.

Large heap usage causes

- Cache misses
- Frequent page faults
- Memory fragmentation

Hence performance becomes slow.

Optimizations

1. Reduce number of threads using Thread Pool.
2. Reduce heap allocation and reuse memory.

Program

```
#include <stdio.h>
```

```
#include <stdlib.h>

int main(){

    int *arr;

    arr = (int*)malloc(1000000*sizeof(int));

    if(arr==NULL) {

        printf("Memory Allocation Failed");

        return 0;

    }

    printf("Memory Allocated Successfully\n");

    free(arr);

    printf("Memory Released\n");

    return 0;

}
```

Q4. Starvation Debugging

Debugging

P5 belongs to

Q3 (Batch Queue)

Higher priority queues

Q1

Q2

always receive CPU first.

Therefore,

P5 keeps waiting indefinitely.

This is called **Starvation**.

Optimization

Implement

Aging

If P5 waits for a long time,

Increase its priority gradually.

Eventually,

P5 moves to Q2 or Q1 and gets CPU time.

Program

```
#include <stdio.h>
```

```
int main() {
```

```
    int priority = 3;
```

```
    printf("Initial Priority = %d\n",priority);
```

```
    for(int i=1;i<=3;i++) {
```

```

    priority--;

    printf("Aging Applied -> Priority = %d\n",priority);

}

printf("Process gets CPU now.\n");

return 0;

}

```

Q5. Context-Switch Overhead Debugging (10 Marks)

Debugging

Given

Context Switches = 20,000/sec

Quantum = 1 ms

Burst Time = 2 ms

Every process is interrupted after only **1 ms**.

Hence,

Almost every process requires another context switch before completion.

Very frequent I/O also forces additional context switches.

Therefore,

CPU spends significant time saving and restoring process states instead of executing processes.

Optimizations

1. Increase Time Quantum (for example, from 1 ms to 4 ms or 5 ms).
2. Reduce unnecessary preemption by using adaptive scheduling or batching I/O operations.

Program

```
#include <stdio.h>

int main(){

    int burst = 2;

    int quantum = 4;

    if(burst <= quantum){

        printf("Process completes without extra context switch.\n");

    }

    else{

        printf("Additional context switch required.\n");

    }

    return 0;

}
```